

技术学习手册

Redis

从 NoSQL 基础到缓存、高可用与安全配置

概念纠偏 · 命令速查 · 原理梳理 · 实战清单

一句话理解 Redis 是以内存为主、提供丰富数据结构与原子操作的键值型数据存储。它常用于缓存、会话、计数、排行榜、限流和实时状态，但不能把“快”误解为“永不丢数据”或“适合所有数据”。

适合：开发学习、面试复习、上线前检查与日常运维速查

整理日期：2026 年 7 月

目录与阅读路线

章节	主题	读完能解决什么
01	认识 NoSQL	澄清“非结构化、无关联、BASE、内存存储”等常见误解
02	认识 Redis	定位、特征、适用场景与边界
03	数据结构与建模	类型、命令、键设计与编码原则
04	缓存与过期	TTL、淘汰策略、缓存风险与治理
05	持久化与高可用	RDB、AOF、复制、Sentinel、Cluster
06	原子性与并发控制	事务、WATCH、Lua、分布式锁
07	配置与安全	配置项释义、生产安全基线
08	运维与排障	观测指标、慢查询、备份与排查顺序
09	实践清单	从安装到上线的练习与验收

阅读建议 第一次学习按顺序阅读；准备面试时重点看第 2、3、4、5、6 章；准备上线时重点看第 7、8、9 章。

先纠正四句话

- NoSQL 更准确地说是 “Not Only SQL”，并不等同于“没有结构”。文档、列族、图和键值数据库都有自己的结构。
- NoSQL 不等于“没有关系”。关系可以由应用、嵌套文档、集合、边或索引表达，只是不一定使用关系表与 JOIN。
- NoSQL 不天然等于 BASE。不同产品、拓扑和操作可提供不同的一致性与事务保证。
- NoSQL 不等于只存内存。Redis 以内存为主；MongoDB、HBase 等通常以磁盘/持久化存储为核心。

1 认识 NoSQL

1.1 概念

NoSQL 是一组非关系型数据管理方案的统称，通常强调灵活的数据模型、按访问模式建模、易于水平扩展，以及针对特定工作负载的性能。它不是一条统一的技术路线，也不是 SQL 的简单反义词。

维度	关系型数据库	NoSQL（概括）
数据模型	表、行、列，模式通常较明确	键值、文档、列族、图等；模式可灵活
关联方式	主外键、JOIN、约束	应用侧聚合、嵌套、集合或图关系
查询	SQL 与成熟查询优化器	产品专用 API/查询语言；能力差异较大
事务	通常强调 ACID 与强约束	从单键原子性到多记录事务不等；不能一概而论
扩展	可垂直扩展，也支持分库分表/分布式	许多产品原生考虑分片与水平扩展
典型选择依据	复杂查询、强约束、通用事务	明确访问模式、高吞吐、低延迟或特殊关系模型

1.2 四类常见 NoSQL

类型	代表	适合的访问方式	典型场景
键值	Redis、DynamoDB	按 key 直接读取/更新	缓存、会话、计数、状态
文档	MongoDB、Couchbase	按字段查询 JSON-like 文档	内容、商品、用户画像
列族	HBase、Cassandra	按行键与列族访问海量稀疏数据	时序、日志、宽表
图	Neo4j、JanusGraph	遍历节点与边	社交关系、推荐、风控关系网

认识NoSQL

	SQL	NoSQL
数据结构	结构化 (Structured)	非结构化
数据关联	关联的 (Relational)	无关联的
查询方式	SQL查询	非SQL
事务特性	ACID	BASE
存储方式	磁盘	内存
扩展性	垂直	水平
使用场景	1) 数据结构固定 2) 相关业务对数据安全性、一致性要求较高	1) 数据结构不固定 2) 对一致性、安全性要求不高 3) 对性能要求

#1 键值类型 (Redis)

#2 文档类型 (MongoDB)

#3 列类型 (HBase)

#4 Graph类型 (Neo4j)

图 1 原始笔记中的 SQL / NoSQL 对比（其中“非结构化、无关联、内存、BASE”应视为概括而非绝对定义）

图表解读 这张图适合用于建立直觉，但选型时应回到具体产品：数据模型、事务范围、一致性、查询能力、持久化介质和扩展方式都可能不同。

- 先写清最频繁的读写路径，再决定是关系表、文档、键值还是图模型。
- 不要只看峰值吞吐，还要评估数据正确性、运维复杂度、备份恢复和团队经验。
- 同一系统可以组合使用 SQL 与 NoSQL，让每种存储承担它擅长的访问模式。

2 认识 Redis

2.1 定位与历史

Redis 全称 Remote Dictionary Server，由 Salvatore Sanfilippo（antirez）在 2009 年启动。它是一个以内存为主的数据结构服务器，通过网络协议提供键值访问，同时支持持久化、复制、集群、发布订阅、脚本等能力。

认识Redis

Redis 诞生于 2009 年全称是 **Remote Dictionary Server**，远程词典服务器，是一个基于内存的键值型 NoSQL 数据库。

特征：

- 键值（key-value）型，value 支持多种不同数据结构，功能丰富
- 单线程，每个命令具备原子性
- 低延迟，速度快（基于内存、IO 多路复用、良好的编码）。
- 支持数据持久化
- 支持主从集群、分片集群
- 支持多语言客户端



图 2 原始笔记：Redis 的定位、特征与发起者

2.2 核心特征（带边界）

- **丰富数据结构**：不仅是 String，还包括 Hash、List、Set、Sorted Set、Stream、Bitmap、HyperLogLog、地理位置等。
- **低延迟**：热点数据驻留内存，协议和数据结构实现高效；实际延迟仍受大 key、慢命令、网络、持久化抖动和系统负载影响。
- **命令原子性**：单条命令通常原子执行；多条命令需要事务、Lua 或专门的原子命令组合。
- **“单线程”需加引号**：核心命令执行路径长期以单线程串行为主，避免锁竞争；网络 I/O、持久化、释放内存等工作可能由其他线程或后台进程承担。
- **可持久化**：支持 RDB 快照与 AOF 日志，但持久化策略决定可接受的数据丢失窗口和恢复速度。
- **可扩展与高可用**：复制用于副本，Sentinel 用于非分片主从故障转移，Cluster 用于分片与故障转移。

2.3 适用与不适用场景

适合	原因	不宜直接使用 / 需谨慎
缓存、会话、验证码	TTL、低延迟、原子更新	唯一事实来源且不能容忍任何丢失
计数器、限流器	INCR、过期、Lua	需要复杂跨表约束与临时分析查询
排行榜、延迟队列	Sorted Set 可按分值排序	单个对象远超可接受内存或网络包大小
实时事件/消费	Stream 支持消费组与确认	需要完整消息中间件语义却未评估重试、堆积和运维

2.4 采用 Redis 前先问

- 数据是否允许丢失？若不允许，在线数据库、持久化、复制和备份各承担什么责任？
- 峰值容量能否放入内存，增长速度如何，达到 `maxmemory` 后允许淘汰哪些键？
- 访问是否主要按 `key`，还是需要复杂查询、强约束与跨记录事务？
- 故障时客户端如何超时、重试、降级与恢复，是否会形成回源风暴？

3 数据结构与建模

3.1 常用数据类型速查

类型	核心特征	常用命令	典型场景
String	字符串、整数或二进制；最大单值不代表推荐大对象	SET / GET / MGET / INCR / SETNX	缓存、计数、锁标记
Hash	一个 key 下存多个字段	HSET / HGET / HMGET / HINCRBY	对象属性、购物车
List	有序、可从两端操作	LPUSH / RPUSH / LPOP / BLPOP	简单队列、时间线
Set	无序去重集合	SADD / SISMEMBER / SINTER	标签、共同关注、去重
Sorted Set	成员唯一，按 score 排序	ZADD / ZRANGE / ZRANK / ZINCRBY	排行榜、延迟任务
Stream	追加日志、ID、消费组	XADD / XREADGROUP / XACK	事件流、工作队列
Bitmap	对 String 进行位操作	SETBIT / GETBIT / BITCOUNT	签到、活跃状态
HyperLogLog	近似基数统计，省内存	PFADD / PFCOUNT / PFMERGE	UV 去重估算
Geo	经纬度索引（底层基于有序集合）	GEOADD / GEOSEARCH	附近的人/门店

选型口诀 只取一个值用 **String**；一个对象多字段用 **Hash**；两端进出用 **List**；去重与集合运算用 **Set**；按分数排序用 **Sorted Set**；需要消费组与确认用 **Stream**。

3.2 键设计原则

- 命名可读且有层次：service:entity:id:field，例如 shop:user:42:profile。冒号只是约定，不会自动形成目录。
- 控制 key 和 value 大小；大 key 会增加网络传输、阻塞、复制和持久化压力。删除大 key 时优先评估 UNLINK。
- 缓存 key 应有 TTL，并加入适度随机抖动，避免同一时刻大规模失效。
- 避免用 KEYS pattern 在线扫描生产库；使用 SCAN 分批遍历，并理解它可能返回重复项。
- Cluster 中需要多 key 原子操作时，用哈希标签让相关 key 落到同一槽，例如 order:{1001}:header 与 order:{1001}:items。

3.3 命令示例

```
# 字符串与过期：原子地写入 60 秒
SET session:8f2a "user:42" EX 60
TTL session:8f2a

# Hash：更新用户字段
HSET user:42 name "Aron" level 3
HINCRBY user:42 level 1

# Sorted Set：排行榜
ZADD rank:weekly 1280 user:42
ZREVRANGE rank:weekly 0 9 WITHSCORES

# 安全遍历（应用端循环 cursor，直到返回 0）
SCAN 0 MATCH user:* COUNT 100
```

3.4 命令使用注意

- 先用 `TYPE` 确认键类型；`WRONGTYPE` 通常意味着键命名冲突或数据模型漂移。
- 批量读取要限制条数和响应体大小；`Pipeline` 能减少网络往返，但不会让整批命令自动原子化。
- `FLUSHALL`、`FLUSHDB`、`CONFIG`、`DEBUG` 等管理命令应限制权限，并避免在业务账号中开放。

4 缓存、过期与内存治理

4.1 TTL 与过期语义

Redis 通过惰性删除与周期性过期扫描回收过期键。TTL 到点并不意味着物理内存会在同一微秒释放，但键在逻辑上应不再可见。覆盖写入、重命名或某些结构操作可能改变 TTL，编码时应显式验证。

```
SET cache:product:1001 "{...}" EX 300
TTL cache:product:1001      # 秒级剩余时间
PTTL cache:product:1001     # 毫秒级剩余时间
EXPIRE cache:product:1001 600
PERSIST cache:product:1001 # 移除过期时间（谨慎使用）
```

4.2 三类缓存风险

问题	现象	治理方式
缓存穿透	大量查询不存在的数据，持续打到数据库	缓存空值（短 TTL）、布隆过滤器、参数校验、限流
缓存击穿	单个热点 key 失效瞬间并发回源	互斥重建、逻辑过期、提前刷新、请求合并
缓存雪崩	大量 key 同时失效或 Redis 整体不可用	TTL 随机抖动、分批预热、隔离降级、多级缓存、高可用
数据不一致	缓存与数据库看到不同版本	Cache-Aside、先更新数据库再删缓存、重试/消息补偿、合理 TTL

4.3 maxmemory 与淘汰策略

当内存达到 maxmemory 后，Redis 按 maxmemory-policy 决定拒绝写入还是淘汰键。策略选择必须与数据是否设置 TTL、是否允许淘汰相匹配。

除下表策略外，volatile-ttl 会在设置了 TTL 的键中优先淘汰剩余生存时间较短的键，适合业务 TTL 能表达回收优先级的场景。

策略家族	含义	适用提示
noeviction	不淘汰；可能返回写入错误	数据不能被自动删除时使用，并配合容量告警
LRU / LFU 家族	allkeys 在全部键中淘汰； volatile 只在设置 TTL 的键中淘汰	通用缓存常用 allkeys ；混合数据需谨慎评估 volatile
allkeys-random / volatile-random	随机淘汰	访问分布均匀或对精确冷热不敏感

容量原则 不要把 **maxmemory** 设到机器物理内存上限。还要给操作系统、复制缓冲区、客户端缓冲区、持久化写时复制和内存碎片留余量。

4.4 内存治理流程

- 按键数量、平均 **value** 大小、数据结构编码和增长率估算基线，再用样本数据实测。
- 为复制、AOF/RDB、重写或快照期间的写时复制、连接缓冲区和碎片预留空间。
- 设置 **maxmemory**、淘汰策略与分级告警；压测达到阈值时的业务行为。
- 持续识别 **big key**、**hot key**、无 **TTL** 缓存和异常增长，并形成容量扩展计划。

5 持久化与高可用

5.1 RDB 与 AOF

机制	工作方式	优势	主要代价 / 风险
RDB	按条件生成某一时点快照	文件紧凑、全量恢复快、适合备份	两次快照间的写入可能丢失；fork 与写时复制有成本
AOF	记录写命令并按策略刷盘，可重写压缩	数据丢失窗口通常更小、过程更可追溯	文件与恢复成本可能更高；刷盘策略影响性能与耐久性
组合使用	同时保留快照和追加日志	兼顾备份、恢复与耐久性	资源占用和运维复杂度更高，必须演练恢复

关键认知 “开启持久化”不等于“零数据丢失”。需要同时定义 RPO（最多允许丢多少数据）、RTO（多久恢复）、备份保留、异地副本和恢复演练。

5.2 复制、Sentinel 与 Cluster

能力	解决什么	不能替代什么
主从复制	读扩展、副本与故障恢复基础	不能自动保证故障切换，也不能替代备份
Sentinel	监控、主观/客观下线判断、非分片主从自动切换、发现服务	不提供数据分片；切换期间可能短暂不可用或丢失未复制写入
Cluster	16384 槽分片、节点间路由、分片环境的故障转移	不自动解决跨槽多 key 操作、全局事务与应用幂等

5.3 选择路径

1. 单机开发/测试：单实例即可，明确数据可丢失性。
2. 容量可由单主承担但要自动故障切换：主从 + Sentinel。
3. 单机内存/吞吐不足，需要水平分片：Redis Cluster；客户端必须支持重定向与拓扑刷新。
4. 无论哪种拓扑：备份应独立于在线副本，定期验证能否在隔离环境恢复。

6 原子性、事务与并发控制

6.1 原子命令优先

能用单条原子命令完成的操作，不要先 **GET** 再 **SET**。例如计数使用 **INCR**，集合成员添加使用 **SADD**，条件写入使用 **SET ... NX/XX**。这样能减少竞态窗口与网络往返。

```
# 仅当 key 不存在时写入，并设置 30 秒过期
SET job:lock:42 "owner-token" NX EX 30

# 原子计数
INCR api:rate:user:42:20260701
EXPIRE api:rate:user:42:20260701 60
```

6.2 MULTI / EXEC 与 WATCH

MULTI/EXEC 会将命令排队后连续执行，但它不是传统关系数据库事务：通常不会因某条运行时错误自动回滚已执行命令。**WATCH** 提供乐观锁：被观察 **key** 在 **EXEC** 前发生变化时，事务不会执行，客户端应重试。

6.3 Lua 脚本

Lua 可把多步读改写合并为服务端原子执行，适合限流、条件扣减和安全释放锁。脚本必须短小、确定且避免长时间阻塞。

```
-- 仅当锁仍属于当前请求时删除
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
else
    return 0
end
```

6.4 分布式锁边界

- 锁值必须是每次请求唯一的 **token**；释放时同时比较 **token**，避免误删他人的锁。
- 锁必须设置过期时间，但过期时间过短会导致任务未完成锁已失效；需要续期或业务幂等。
- 主从复制通常是异步的，故障切换可能让刚写入的锁丢失；高风险互斥需评估一致性模型与 **fencing token**。
- 锁不是数据正确性的最终防线。数据库唯一约束、版本号、幂等键和状态机往往更可靠。

7 配置与安全

7.1 原始配置截图中的风险



图 3 原始笔记中的 bind / daemonize / requirepass 示例



图 4 原始笔记中的端口、目录、数据库数、内存和日志配置

重要纠正 bind 0.0.0.0 会监听所有 IPv4 接口；它本身不是“允许任意 IP 安全访问”，而是扩大暴露面。弱密码 123321 不应使用。生产环境应优先网络隔离、防火墙/安全组、protected-mode、ACL、TLS（如适用）和强凭据。

7.2 配置项释义与建议

配置项	含义	建议
bind	监听的本地接口	默认只监听回环或明确的私网地址；不要为了省事直接暴露公网
protected-mode	在缺少安全配置时限制外部访问	保持开启；它不是网络隔离的替代品
port	TCP 端口，默认常见为 6379	端口修改只能降低噪声，不能替代认证与防火墙
daemonize	是否在后台运行	容器/systemd 下通常前台运行，由进程管理器接管
dir / dbfilename	持久化文件目录与名称	使用持久卷，设置正确权限，并监控空间
databases	逻辑数据库数量（非 Cluster 常见）	逻辑 DB 不提供权限/资源隔离；生产隔离优先独立实例
maxmemory	数据与部分开销的内存控制阈值	按容量评估设置，并搭配 maxmemory-policy
logfile	日志输出位置	容器中常输出到标准输出；宿主机部署考虑轮转
requirepass	传统全局密码配置	新部署优先 ACL 用户；不要把明文密码提交到代码仓库

7.3 更安全的示例

```
# 仅示意：地址、目录、内存和策略应按环境调整
bind 127.0.0.1 10.0.10.15
protected-mode yes
port 6379
daemonize no

dir /var/lib/redis
dbfilename dump.rdb
appendonly yes

maxmemory 4gb
maxmemory-policy allkeys-lfu

# 建议使用 ACL 配置最小权限用户；凭据由密钥系统注入
# user app on >REDACTED ~app:* +@read +@write -@dangerous
```

ACL 提醒 上面的 ACL 只是思路示例，不能直接照抄到所有业务。先列出应用需要的命令与 key 前缀，再最小化权限；禁用或限制管理型、危险型命令，并单独管理运维账号。

7.4 生产安全基线

- Redis 仅部署在受控网络，不直接暴露公网；使用安全组/防火墙限制来源。
- 启用 ACL 与强凭据，按应用拆分用户和 key 范围；凭据定期轮换。
- 管理面与业务面分离；限制 CONFIG、MODULE、DEBUG、SHUTDOWN、FLUSHALL 等高危命令。
- 需要跨不可信网络时启用 TLS 或使用受控加密隧道，并验证证书。
- 设置 client-output-buffer-limit，防止慢消费者或 Pub/Sub 客户端拖垮内存。
- 持续更新受支持版本，关注安全公告，并在升级前做兼容与回滚演练。

8 运维、观测与排障

8.1 日常观测

关注点	命令 / 指标	异常信号
内存	INFO memory、MEMORY STATS、MEMORY USAGE key	used_memory 接近阈值、碎片异常、evicted_keys 增长
性能	INFO commandstats、SLOWLOG GET、LATENCY DOCTOR	p99 上升、慢命令积累、事件循环被阻塞
客户端	INFO clients、CLIENT LIST	连接数激增、blocked_clients、输出缓冲过大
持久化	INFO persistence	rdb/aof 状态失败、fork 耗时或 COW 内存过高
复制	INFO replication、ROLE	复制延迟、频繁全量同步、主从链路断开
键空间	INFO keyspace、SCAN、--bigkeys（低峰谨慎）	过期率异常、大 key、键数量失控

8.2 排障顺序

1. 确认影响：哪些接口、实例、分片、时间范围与百分位延迟受影响。
2. 检查资源：CPU、内存、磁盘延迟/空间、网络、连接数与宿主机抖动。
3. 检查 Redis：慢日志、阻塞客户端、big key、hot key、持久化与复制状态。
4. 检查变更：代码发布、配置、数据导入、备份/重写、扩缩容是否同时发生。
5. 先止损再定位：限流、降级、切流、扩容或暂停高风险任务；保留现场指标。
6. 复盘并固化：补告警、容量阈值、压测用例、变更检查和恢复演练。

8.3 常见性能陷阱

- **O(N) 命令：**KEYS、全量集合读取、对超大集合的聚合可能长时间阻塞。先看命令复杂度与数据规模。
- **大 key：**一次读写、删除、复制或迁移占用过多时间与带宽。拆分数据并限制单次返回量。
- **热 key：**流量集中到单键/单分片，整体容量仍有余量但局部打满。考虑本地缓存、读副本、拆 key 或请求合并。
- **无限队列：**List/Stream/PubSub 消费跟不上导致内存持续增长。设置长度策略、积压告警与背压。
- **连接风暴：**每次请求新建连接，握手与文件描述符耗尽。使用有界连接池与合理超时。

9 实践路线与上线清单

9.1 建议练习

1. 启动单实例，用 `redis-cli` 完成 String、Hash、List、Set、Sorted Set、Stream 的基本操作。
2. 实现一个带 TTL 的 Cache-Aside 商品缓存，模拟缓存穿透、击穿与雪崩并加入治理。
3. 用 INCR + EXPIRE 或 Lua 实现固定窗口限流，并测试并发下的边界。
4. 开启 RDB/AOF，写入数据后模拟重启，记录恢复时间与可接受的数据丢失窗口。
5. 搭建主从 + Sentinel，主动停止主节点，观察切换、客户端重连与数据一致性。
6. 制造 big key、慢命令与连接堆积，使用 INFO、SLOWLOG、LATENCY、MEMORY 定位。

9.2 上线前检查

检查项	验收标准
容量	已估算键数量、平均/峰值对象大小、复制/持久化开销和增长速度
内存	已设置 <code>maxmemory</code> 、淘汰策略和阈值告警，并保留系统余量
网络	未暴露公网；来源网段、端口、TLS/隧道策略明确
认证	ACL 最小权限、强凭据、密钥托管与轮换机制已落实
高可用	拓扑与业务目标匹配；客户端支持重连、重试、拓扑刷新和幂等
持久化	RPO/RTO 明确；配置、备份、异地保存和恢复演练均通过
缓存设计	TTL、抖动、空值、回源保护、降级和一致性补偿已实现
键设计	无未评估的大 key/热 key；禁止在线 KEYS；批量操作有上限
观测	延迟、命中率、内存、淘汰、连接、慢日志、持久化、复制均有告警
变更	升级、扩缩容、故障切换、数据迁移均有回滚步骤

最终原则 先定义数据正确性与可用性目标，再选择 Redis 的数据结构、持久化、拓扑和安全策略。不要从“Redis 很快”倒推出“什么都放 Redis”。

附录 A 常用命令速查

类别	命令	说明
连接	PING / AUTH / HELLO	连通性、认证与协议协商
键	EXISTS / TYPE / EXPIRE / TTL / UNLINK	存在、类型、过期与异步删除
字符串	SET / GET / MGET / INCRBY	值、批量读、计数
Hash	HSET / HGETALL / HSCAN / HINCRBY	对象字段与渐进遍历
List	LPUSH / RPOP / BLPOP / LRANGE	队列与区间读取
Set	SADD / SREM / SISMEMBER / SINTER	去重与集合运算
Sorted Set	ZADD / ZRANGE / ZSCORE / ZPOPMIN	排序与优先队列
Stream	XADD / XREADGROUP / XACK / XPENDING	事件、消费组与待处理消息
事务/脚本	MULTI / EXEC / WATCH / EVAL	多命令编排与原子脚本
诊断	INFO / SLOWLOG / LATENCY / MEMORY / CLIENT	运行状态、延迟、内存与连接

附录 B 术语表

术语	含义
ACID	原子性、一致性、隔离性、持久性；具体保证取决于系统与操作范围
BASE	基本可用、软状态、最终一致；一种分布式系统设计取向，不是全部 NoSQL 的标签
RPO	恢复点目标：故障后最多可接受丢失多少数据
RTO	恢复时间目标：故障后多久恢复服务
TTL	Time To Live，键剩余生存时间
Cache-Aside	应用先查缓存，未命中查数据库并回填；更新时通常更新数据库后删除缓存
Big Key / Hot Key	值或成员很多的键 / 承受异常集中流量的键
Eviction	达到内存阈值后按策略淘汰键